TRIE (PREFIX TREE)

FAANG Master Guide — Java Edition

💡 WHY TRIES?

A HashMap stores full string keys, but cannot search prefixes efficiently (O(N \cdot L)). A Trie stores characters as nodes, enabling **O(L) prefix lookup** independent of dictionary size N!

💡 THE SHARED PREFIX AHA

Strings sharing the same prefix (e.g. "cat", "cater", "cats") share the exact same branch nodes `c → a → t`, saving space and enabling instant autocomplete!

1 CORE NODE STRUCTURE

```
public class TrieNode {
  TrieNode[] children = new TrieNod
e[26];
  boolean isWord = false;
  // Optional for Word Search II:
  String word = null;
}
```

2 VISUALIZING TRIE BRANCHING

Inserting "app" and "apple":

```
root
 └─ 'a'
     └─ 'p'
         └─ 'p' (isWord = true: "ap
p")
             └─ 'l'
                 └─ 'e' (isWord = t
rue: "apple")
```

📚 PREREQUISITES & COMPLEXITY REASONING

Operation	Time Complexity	Auxiliary Space
Insert Word (Length L)	**O(L)**	**O(L) nodes**
Search Exact Word (Length L)	**O(L)**	**O(1)**
StartsWith Prefix (Length L)	**O(L)**	**O(1)**
Bitwise 31-Bit Max XOR	**O(31) = O(1)**	**O(31 · N) nodes**

CLUES THAT SCREAM TRIE

1. *"Autocomplete / Search suggestions system"* \rightarrow Standard Trie
2. *"Find words in 2D grid"* \rightarrow Trie + DFS Grid Pruning (Word Search II)
3. *"Replace words with shortest root prefix"* \rightarrow Prefix Match Trie
4. *"Maximum XOR of two numbers"* \rightarrow 31-Bit Binary Trie

1 STANDARD IMPLEMENT TRIE CLASS (LC 208)

```
public class Trie {
  private class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isWord = false;
  }
  private final TrieNode root = new TrieNode();

  public void insert(String word) {
    TrieNode curr = root;
    for (char c : word.toCharArray()) {
      int idx = c - 'a';
      if (curr.children[idx] == null) curr.children[idx] = new TrieNode();
      curr = curr.children[idx];
    }
    curr.isWord = true;
  }

  public boolean search(String word) {
    TrieNode node = find(word);
    return node != null && node.isWord;
  }

  public boolean startsWith(String prefix) {
    return find(prefix) != null;
  }

  private TrieNode find(String str) {
    TrieNode curr = root;
    for (char c : str.toCharArray()) {
      int idx = c - 'a';
      if (curr.children[idx] == null) return null;
      curr = curr.children[idx];
    }
    return curr;
  }
}
```

PATTERN 1 ADD AND SEARCH WORDS WITH WILDCARD '.' (Trie + Backtracking DFS)

e.g. Design Add and Search Words Data Structure (LC 211)

WILDCARD DOT MECHANICS

When character is `.`, loop through ALL 26 children. If ANY child branch matches the remaining string, return true!

TEMPLATE — LC 211

```
public boolean search(String word) {
  return searchInNode(word, 0, root);
}
private boolean searchInNode(String word, int index, TrieNode node) {
  if (node == null) return false;
  if (index == word.length()) return node.isWord;
  char c = word.charAt(index);
  if (c == '.') {
    for (TrieNode child : node.children) {
      if (child != null && searchInNode(word, index + 1, child)) return true;
    }
    return false;
  } else {
    return searchInNode(word, index + 1, node.children[c - 'a']);
  }
}
```

PATTERN 2 SHORTEST ROOT PREFIX REPLACEMENT e.g. Replace Words (LC 648)

WHEN TO USE

Replace each word in a sentence with the shortest matching root in dictionary.

TEMPLATE — LC 648

```
public String replaceWords(List<String> dictionary, String sentence) {
  Trie trie = new Trie();
  for (String root : dictionary) trie.insert(root);
  StringBuilder sb = new StringBuilder();
  for (String word : sentence.split(" ")) {
    if (sb.length() > 0) sb.append(" ");
    sb.append(trie.findRoot(word));
  }
  return sb.toString();
}
// Inside Trie:
public String findRoot(String word) {
  TrieNode curr = root; StringBuilder prefix = new StringBuilder();
  for (char c : word.toCharArray()) {
    if (curr.children[c - 'a'] == null || curr.isWord) break;
    prefix.append(c); curr = curr.children[c - 'a'];
  }
  return curr.isWord ? prefix.toString() : word;
}
```

PATTERN 3 AUTOCOMPLETE SEARCH SUGGESTIONS e.g. Search Suggestions System (LC 1268)

TOP 3 LEXICOGRAPHICAL MATCH

Store top 3 suggestions list at each `TrieNode` during insertion!

TEMPLATE — LC 1268

```
class SugNode {
  SugNode[] children = new SugNode[26];
  List<String> top3 = new ArrayList<>();
}
void insert(String w) {
  SugNode curr = root;
  for (char c : w.toCharArray()) {
    int i = c - 'a';
    if (curr.children[i] == null) curr.children[i] = new SugNode();
    curr = curr.children[i];
    if (curr.top3.size() < 3) curr.top3.add(w);
  }
}
```

PATTERN 4 WORD SEARCH II (Trie + 2D Grid DFS Pruning) e.g. Word Search II (LC 212)

WHY TRIE BEATS PLAIN DFS

Searching M words with plain DFS runs grid DFS M times (O(M \cdot 4^L) TLE!). Inserting all words into a Trie lets a single grid DFS search all words simultaneously while pruning invalid branches instantly!

TEMPLATE — LC 212

```
public List<String> findWords(char[][] board, String[] words) {
  TrieNode root = new TrieNode();
  for (String w : words) {
    TrieNode curr = root;
    for (char c : w.toCharArray()) {
      if (curr.children[c - 'a'] == null) curr.children[c - 'a'] = new TrieNo
de();
      curr = curr.children[c - 'a'];
    }
    curr.word = w; // Store full word at leaf node!
  }
  List<String> res = new ArrayList<>();
  for (int r = 0; r < board.length; r++)
    for (int c = 0; c < board[0].length; c++)
      dfs(board, r, c, root, res);
  return res;
}
private void dfs(char[][] b, int r, int c, TrieNode node, List<String> res) {
  char ch = b[r][c];
  if (ch == '#' || node.children[ch - 'a'] == null) return;
  node = node.children[ch - 'a'];
  if (node.word != null) { res.add(node.word); node.word = null; } // Avoid d
ups
  b[r][c] = '#'; // Mark visited
  for (int[] d : DIRS) {
    int nr = r + d[0], nc = c + d[1];
    if (nr>=0 && nr=0 && nc
```

PATTERN 5 BITWISE 31-BIT BINARY TRIE (Maximum XOR of Two Numbers)

e.g. Maximum XOR of Two Numbers in an Array (LC 421)

GREEDY OPPOSITE BIT SELECTION

For each bit from position 30 down to 0, greedily traverse the OPPOSITE bit (`1 ^ bit`) to maximize XOR value!

TEMPLATE — LC 421

```
class BitNode { BitNode[] child = new BitNode[2]; }

public int findMaximumXOR(int[] nums) {
  BitNode root = new BitNode();
  for (int num : nums) {
    BitNode curr = root;
    for (int i = 30; i >= 0; i--) {
      int bit = (num >> i) & 1;
      if (curr.child[bit] == null) curr.child[bit] = new BitNode();
      curr = curr.child[bit];
    }
  }
  int maxXor = 0;
  for (int num : nums) {
    BitNode curr = root; int currXor = 0;
    for (int i = 30; i >= 0; i--) {
      int bit = (num >> i) & 1;
      int opp = 1 ^ bit;
      if (curr.child[opp] != null) {
        currXor |= (1 << i);
        curr = curr.child[opp];
      } else {
        curr = curr.child[bit];
      }
    }
    maxXor = Math.max(maxXor, currXor);
  }
  return maxXor;
}
```

TRIE (PREFIX TREE)

Decision Tree & Trigger Words

TRIE — WHICH PATTERN TO USE?

Start: What is the input type & operation?

- 🔤 Exact prefix search / autocomplete → **Standard Trie** `child array
- ❓ Wildcard search '.' in words → **Trie + DFS** Loop children on `.`
- 🧩 Multi-word search in 2D Grid → **Trie + Grid DFS** Store `node.word` at leaf
- ⚡ Max XOR of pair of integers → **Bitwise Binary Trie** 31-Bit binary branches

🎯 TRIGGER WORDS DIRECTORY

Trigger Word	Variant	Action
"Prefix search", "Autocomplete"	Standard Trie	Traverse `children[c - 'a']`
"Wildcard dot search"	Trie + DFS	Recurse on non-null children
"Word Boggle", "Grid multi-word"	Trie + Grid DFS	Prune grid DFS using Trie nodes
"Maximum XOR"	31-Bit Binary Trie	Greedily pick opposite bit

💼 INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Trie vs HashMap for words?	HashMap is O(1) for full matches, but CANNOT do prefix searches. Trie does prefix lookup in O(L)!
Array `children[26]` vs HashMap?	Array `children[26]` is faster with O(1) index access for lowercase English. HashMap is better for large Unicode alphabets.

EASY & MEDIUM — Core Trie Operations

LC 208 · Implement Trie — MEDIUM

Pattern: Standard Trie
Key Insight: Insert & find using `children[26]`.

```
class Trie {
  TrieNode root = new TrieNode();
  public void insert(String w) {
    TrieNode c = root;
    for (char ch : w.toCharArray()) {
      if (c.children[ch-'a']==null) c.
children[ch-'a']=new TrieNode();
      c = c.children[ch-'a'];
    }
    c.isWord = true;
  }
}
```

LC 211 · Add & Search Words — MEDIUM

Pattern: Wildcard Trie DFS
Key Insight: Recurse on all 26 children for `.`.

```
private boolean search(String w, int i
dx, TrieNode n) {
  if (n == null) return false;
  if (idx == w.length()) return n.isWo
rd;
  char c = w.charAt(idx);
  if (c == '.') {
    for (TrieNode child : n.children)
      if (child ≠ null && search(w, i
dx + 1, child)) return true;
    return false;
  }
  return search(w, idx + 1, n.children
[c - 'a']);
}
```

MEDIUM — Prefix Replacement & Map Sum

LC 648 · Replace Words — MEDIUM

Pattern: Prefix Match Trie
Key Insight: Break loop early when `curr.isWord == true`.

```
public String findRoot(String word) {
  TrieNode c = root; StringBuilder pref
= new StringBuilder();
  for (char ch : word.toCharArray()) {
    if (c.children[ch - 'a'] == null ||
c.isWord) break;
    pref.append(ch); c = c.children[ch -
'a'];
  }
  return c.isWord ? pref.toString() : wo
rd;
}
```

LC 1268 · Search Suggestions — MEDIUM

Pattern: Autocomplete Top 3
Key Insight: Store top 3 strings list at each node.

```
void insert(String w) {
  SugNode c = root;
  for (char ch : w.toCharArray()) {
    int i = ch - 'a';
    if (c.children[i] == null) c.childre
n[i] = new SugNode();
    c = c.children[i];
    if (c.top3.size() < 3) c.top3.add
(w);
  }
}
```

HARD — Grid DFS Pruning

LC 212 · Word Search II — HARD

Pattern: Trie + Grid DFS
Key Insight: Store full word at leaf. Prune DFS.

```
private void dfs(char[][] b, int r, int
c, TrieNode n, List<String> res) {
  char ch = b[r][c];
  if (ch == '#' || n.children[ch - 'a'] =
= null) return;
  n = n.children[ch - 'a'];
  if (n.word != null) { res.add(n.word);
n.word = null; }
  b[r][c] = '#';
  for (int[] d : DIRS) {
    int nr = r + d[0], nc = c + d[1];
    if (nr>=0 && nr=0 && nc
```

HARD — Bitwise Maximum XOR

LC 421 · Maximum XOR of Two Numbers — MEDIUM

Pattern: 31-Bit Binary Trie
Key Insight: Greedily pick opposite bit `1 ^ bit`.

```
for (int i = 30; i >= 0; i--) {
  int bit = (num >> i) & 1, opp = 1 ^ bi
t;
  if (curr.child[opp] != null) {
    currXor |= (1 << i); curr = curr.chil
d[opp];
  } else curr = curr.child[bit];
}
```

COMPLETE TRIE PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
208	Implement Trie	Standard Trie	O(L)	O(L · N)	M
211	Add & Search Words	Wildcard Trie DFS	O(L) / O(26^L)	O(L · N)	M
648	Replace Words	Prefix Match Trie	O(N · L)	O(L · N)	M
1268	Search Suggestions System	Autocomplete Top 3	O(L)	O(L · N)	M
212	Word Search II	Trie + Grid DFS	O(R · C · 4^L)	O(L · N)	H
421	Maximum XOR of Two Numbers	31-Bit Binary Trie	O(31 · N)	O(31 · N)	M

TRIE (PREFIX TREE)

Dry Run, Math Proofs & Cheat Sheet

🔍 DRY RUN — Trie Insertion ("app")

Char	Idx (`c - 'a'`)	Child Exists?	Action / Node State
'a'	0	No	Create `children[0]`, move `curr = children[0]`
'p'	15	No	Create `children[15]`, move `curr = children[15]`
'p'	15	No	Create `children[15]`, move `curr = children[15]`
End	-	-	Set `curr.isWord = true` ✅

📐 MATHEMATICAL PROOF — O(L) PREFIX SEARCH

Claim: Prefix searching in a Trie takes O(L) time regardless of dictionary size N.

Proof: At each character step $i \in [0, L-1]$, array indexing `children[c - 'a']` takes O(1) time. Moving down L characters takes $L × O(1) = O(L) \rightarrow$ **Strictly O(L) Time!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Implement Trie class from scratch in 3m
- ☐ Write wildcard '.' search with DFS
- ☐ Code Replace Words with shortest root
- ☐ Code Search Suggestions with Top 3 list
- ☐ Write Word Search II (Trie + Grid DFS)
- ☐ Code Bitwise 31-Bit Binary Trie for Max XOR
- ☐ Deduplicate words in Word Search II (`node.word = null`)
- ☐ Prove why Trie search is O(L)

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Avoid Result Duplicates in Word Search II

In Word Search II, set `node.word = null` after adding to result list to prevent inserting duplicate words!

Rule 2 — Greedily Pick Opposite Bit for Max XOR

In Bitwise Binary Trie, try `1 ^ bit` first at each position $i \in [30..0]. If opposite bit child exists, OR (1 \ll i)$ into result!